

CSC 4553

Fall 2026 - Day 10

Admin

- Don't forget to sign up for assignment 1 review
<https://calendar.app.google/71iRFEG6RNLqf9P29>
- Lab due Friday, 18:00
- Quiz due Monday, 18:00
- Exam 1 next Thursday in-lecture

Threads

Questions to consider

- What are the differences between processes and threads?
- When are threads beneficial?
- How are threads limited by Amdahl's Law?

Motivation: context switching is expensive

- consider the modern word processor, which concurrently:
 - accepts user input,
 - performs auto-correct/spell checking;
 - auto-detects formatting
 - auto-save (nothing could be entered until save was complete);
- All impossible (or at least very inconvenient) without threads.

Processes vs threads

- Potentially confusing due to overlapping terminology: we can describe both a **process** and a **thread** as running
- Terminology can be helpful for remembering the distinction:
 - A computing **process** requires multiple resources: the CPU, memory, files, etc.
 - A **thread** of execution abstracts CPU state

Processes vs threads (cont'd)

- Processes *contain* threads; threads *belong* to a process
 - Only one exception: the kernel may have threads of execution not associated with any user process
- A process is considered to be running when one or more of its threads are running
- Different operating systems use different terminology, but share common ideas

Intra-process communication

- Communication between multiple threads in a process is usually accomplished using **shared memory** (since they naturally share it)
 - Potential issues?
 - Synchronization (we'll come back to this)
- Threads within a process share what the program has: (code, heap, globals, files).
- Thread keep their own copies of what they're doing: (registers, stack).
 - Why?

Why do we use threads? Pros

- To support multiple activities within a single process
- **Responsiveness**: a process may continue to run, even if part of it is blocked (depends on thread implementation)
- **Resource sharing**: Process are isolated so can only communicate with help from the kernel; threads share a memory space and a process's resources (e.g. files)
- **Economy**: Allocating resources for multiple processes is costly, because it involves the kernel. Threads are much faster to allocate and destroy. Allows for pop-up threads, dynamically created threads to support load
- **Hardware support**: modern CPUs support multithreading, where threads can run in parallel

Thread cons

- This is hard. Burdens programmer needs to solve:
 - Dividing activities
 - Balance
 - Data splitting
 - Data dependency
 - Testing and debugging
- Share the same address space and global variable: **loss of isolation**. A runaway thread can wipe out other threads
- No built-in mechanism for pre-emption, so threads should be well-behaved (unlike greedy processes)
- n threads see $1/n$ of a single CPU; CPU-bound processes make thread moot, or detrimental

What makes more sense, threads or process?

- Web server
- OS services (think system daemons)
- Web browser

What makes more sense, threads or process?

- Web server - thread (through there are many hybrid implementations)
- OS services (think system daemons) - process
- Web browser - both: each tab is a process, but each tab has multiple threads

Side note: `fork()` and threading

- What happens to a process with multiple threads when you `fork()`?
- Should a child inherit the threads of their parent?
- What happens if two threads are competing for the same resource? E.g., who gets the input from a keyboard or established network connection?
- In reality (Linux): a child process, only the calling thread is replicated.

A word of caution: Amdahl's Law

- Threads are awesome, let's parallelize everything, right?
- Amdahl's Law identifies performance gains from adding additional cores to an application that has both serial and parallel components
 - S is serial portion
 - N processing cores

$$speedup \leq \frac{1}{S + \frac{(1-S)}{N}}$$

- That is, if application is 75% parallel / 25% serial, moving from 1 to 2 cores results in speedup of **1.6×**
- As N approaches infinity, speedup approaches $1/S$ (in reality it's worse)
- **Serial portion has a disproportionate effect on performance**

What isn't clear?

Comments? Thoughts?

Thread models

Questions to consider

- How do user threads and kernel threads relate?
- What are the different kernel thread models and how do they compare?
 - When should you use each?

A tale of two programs

```
1 // spawn 100,000 "threads"
2 for (int i = 0; i < 100000; i++)
3     pthread_create(&t[i], NULL, work, NULL)
```

C: dies, or crawls

```
1 // spawn 100,000 "threads"
2 for i := 0; i < 100000; i++ {
3     go work()
4 }
```

Go: shrugs, finishes in ms

Same machine. Same kernel. Same word “thread.”

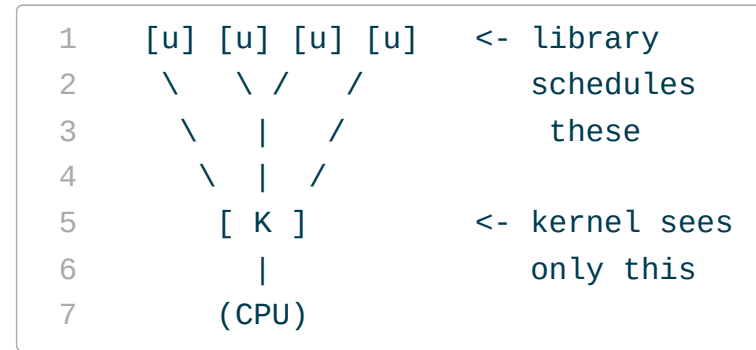
Both are lying to you, differently. Today: *who is lying, and how.*

First things first

- A kernel thread is the unit of execution that the OS can run on a CPU core
- A process can only make progress if at least one of its kernel threads is scheduled on a core
- CPUs run kernel threads, not processes
- The OS scheduler only schedules kernel threads, not user threads
- So: every “thread” you create in *any* language eventually has to borrow one of these

User space threads

- Totally unknown to the OS
- Thread library manages the thread table and scheduling
- Context switch is a function call within the process
 - no TRAP, no kernel, very fast
- User-level threads must “borrow” a kernel thread to run
- So there are *two* schedulers here, and the OS only knows about one of them



Hold onto this shape, we'll give it a formal name in a few slides.

Kernel threads

- Thread libraries can also have kernel support (this is the common case today)
- There is no user space runtime environment or thread table: all is managed within the kernel
- Support for multiple cores
- All thread interfaces are system calls handled by the kernel
- Instead of blocking on a system call, the kernel now has the intelligence to schedule another thread

Kernel threads aren't free

- Creating and destroying threads comes at a higher cost (a kernel TRAP)
- Every thread is a kernel object: a `task_struct`, a kernel stack, a PID, a runqueue entry
- A thread's stack is also reserved up front and fixed: **8 MB** by default (`ulimit -s`)
- What should we do?
 - **Thread pools**: a collection of pre-allocated threads, assigned as needed
 - Pay the cost k times at startup instead of n times at runtime
- Try it: `top -H`, or `ps -eLf | wc -l`, you likely have thousands right now

Thread design models: many-to-one (M:1)

- A single kernel thread supports many user level threads

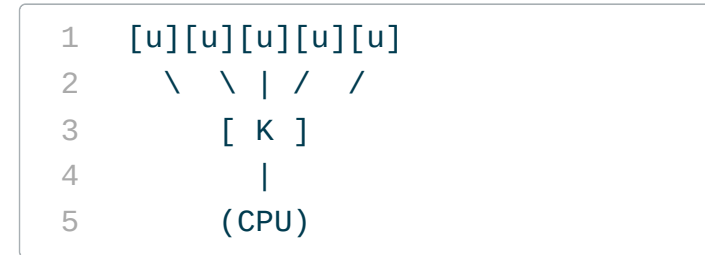
→ This is the pure user-space threading from earlier, under its formal name

- **No parallelism**: one kernel thread, one core, full stop

- Now the bill comes due: one user thread calls `read()` on a cold file

→ **All of them block**: the kernel blocked the *process*; it never heard of the other 99

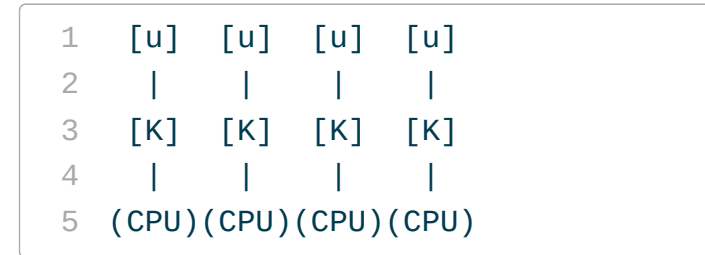
- M:1 can make sense when you need total scheduling control, or have only a minimal hardware footprint (e.g., single-core embedded systems)



One blocked user thread
freezes the whole column.

Thread models: one-to-one (1:1)

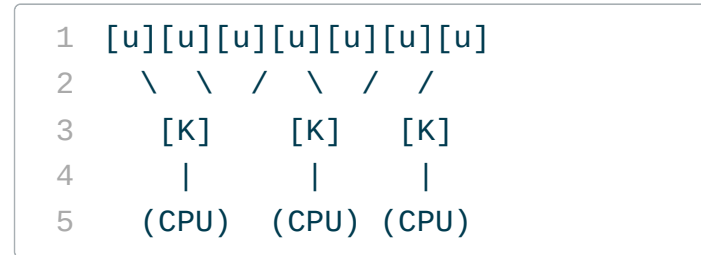
- One kernel thread per user thread (dominant model today)
- Can operate in parallel; if one blocks, others continue
- Downsides?
 - Lose the advantage of fast switching between user threads
 - Any new user thread requires kernel thread creation: *expensive*
 - Programmer loses scheduling control: kernel makes decisions
- Very popular: every `pthread_create` you've ever written
 - Easy to implement, and machines have cores to spare



Simple, parallel, and you pay per thread.

Thread models: many-to-many (M:N)

- Set of user threads multiplexed over a pool of kernel threads
- A user thread isn't bound to a particular kernel thread
 - If a kernel thread blocks while supporting 3 user threads, the others migrate
- Parallelism *and* cheap creation. Best of all worlds, right?
 - Hard to implement without deep language support
 - Two schedulers to reason about (user space and kernel)



Goroutine stacks start at **2 KB**, not 8 MB.

M:N: the pendulum

- The idea keeps getting adopted, abandoned, and re-adopted:
 - **Solaris 2.2** (1993) shipped M:N → a 1:1 library was offered alongside it in Solaris 8, became the default in Solaris 9 (2002), and M:N was retired in that same release
 - **Java** had M:1 “green threads” → replaced by native 1:1 threads, gone from Sun’s JDK in 1.3 (2000)
 - **Go** bet the language on it and won (goroutines, [GOMAXPROCS](#))
 - **Java** brought it back in JDK 21 (Sept 2023) as virtual threads (Project Loom)
- Why the churn? M:N only pays off if *every* blocking call can be caught and turned into a scheduling decision
- Go and Loom can do that: they own their standard libraries

Which models make sense?

- Distributed scientific computing
 - **1:1**: You want full-speed access to hardware, not an extra runtime scheduler in the way
- Embedded system on a single-core MCU
 - **M:1**: Simplicity and determinism matter more than scalability; there's no second core to exploit anyway
- Prometheus collector
 - **M:N**: Scrapes thousands of endpoints concurrently using goroutines; the Go runtime multiplexes them onto a small pool of kernel threads — massive scale without a kernel thread per target

Back to our tale of two programs

```
1 // spawn 100,000 "threads"
2 for (int i = 0; i < 100000; i++)
3     pthread_create(&t[i], NULL, work, NULL)
```

```
1 // spawn 100,000 "threads"
2 for i := 0; i < 100000; i++ {
3     go work()
4 }
```

C: dies, or crawls

Go: shrugs, finishes in ms

- **C:** `pthread_create` is 1:1: 100,000 kernel threads, 100,000 traps, 8 MB stacks each
 - Lie: “a thread is cheap.” It’s a full kernel object
- **Go:** `go` is M:N: 100,000 goroutines on ~**GOMAXPROCS** kernel threads, 2 KB stacks
 - Lie: “you have 100,000 threads.” The kernel sees a handful

The one-slide version

	M:1	1:1	M:N
Parallelism	no	yes	yes
Creation cost	cheap	expensive	cheap
One blocks, all block?	yes	no	no
Who schedules?	you	kernel	both
Seen in	RTOS	Linux, Windows	Go, Java Loom

Tl;dr: there is no winner, only what you're willing to pay for.

What isn't clear?

Comments? Thoughts?

Midterm 1 boundary